BHANU PRATAP RAJAK

FRP Manuscript

Research

Document Details

Submission ID
trn:oid:::3618:140333582

Submission Date
May 25, 2026, 10:55 AM GMT+5:30

Download Date
May 25, 2026, 10:58 AM GMT+5:30

File Name
FRP Manuscript.docx

File Size
391.0 KB

9 Pages

2,417 Words

14,530 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

0 AI-generated only 0%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Automated Detection of Hardcoded Secrets in Modern Source Code Repositories using LLM-Augmented Static Analysis

Dinanath Dash[*,1], Ashutosh Raj[*,2], Avignyan Sarangi[*,3], Satyajeet Satapathy[*,4], Bhanu Pratap Rajak[*,5]

[*]Department of Computer Science and Engineering, Siksha 'O' Anusandhan (Deemed to be) University, Bhubaneswar, Odisha, India

[1]dashdinanath056@gmail.com
[2]ashutosh829484@gmail.com
[3]avignyan.sarangi@gmail.com
[4]satyajeetsat1@gmail.com
[5]bhanupratapprajak@soa.ac.in

Abstract. Static secret scanners based on traditional deterministic regex lack the semantic capacity needed to distinguish between real cryptographic keys and dummy test variables. This blindness leads to extremely high False Positive Rates (FPR), which causes critical alert fatigue in the DevSecOps environment. For overcoming this limitation, we have developed a dual-stage scanning pipeline, where Stage 1 uses Regex as a fast pre-filter for extracting candidates without exhausting the token budget. Stage 2 then retrieves the abstract syntax tree (AST) of the 11-line micro-context and runs semantic checks on it using the 4-bit quantized version of Meta Llama 3 (8B) model. In order to ensure data privacy with a zero-trust approach, the entire pipeline runs locally via Apple MLX on Unified Memory with few-shot prompting stabilizing strict JSON outputs. The evaluation against the benchmark dataset CredData showed that our hybrid model achieves the highest F1-score of 0.8723 (Precision: 0.8117; Recall: 0.9427) on 1,000-hit sample. Structurally outperforming offline baselines such as Gitleaks (Recall: 0.2600) and TruffleHog (Recall: 0.0123), our hybrid model suffered greatly from the 3.5-hour inference bottleneck, confirming the detection hypothesis yet emphasizing the need for batch-processing optimization.

Keywords: DevSecOps, Large Language Models, Static Code Analysis, Zero-Trust Architecture, Parameter-Efficient Fine-Tuning.

1 Introduction

CI/CD pipelines have evolved very rapidly and have thus had a significant impact on the software supply chain. Although the DevSecOps approach tries to address issues related to security testing in the development process, one

significant risk factor is the inadvertent inclusion of hardcoded secrets such as APIs or cryptographic keys.

1.1 Motivations

In contrast, traditional methods of Static Application Security Testing (SAST) depend predominantly on deterministic Regex or strings with high entropy. Old-school tools are purely lacking semantic context analysis, as the analysis is done on isolated strings alone. This creates a situation that results in unacceptably high False Positive Rate (FPR) due to dummy variables, test credentials, and fake information. As a result, alert fatigue is created for the security team, leading to delay in CI/CD processes.

1.2 Objectives

Mainly, the main purpose of this study is to design a two-stage hybrid scanning system that can significantly reduce false positive rates for detecting secrets but without sacrificing the high recall rate required to meet stringent compliance requirements. In addition, this study endeavors to provide data privacy using the concept of zero trust through full offline semantic validation and comparing accuracy and inference delays with traditional offline scanners.

1.3 Original Contributions

This paper introduces the following technical innovations to DevSecOps static analysis:

- Cascading Filter Architecture: A remedial approach to LLM token exhaustion employing deterministic Regex as a pre-filter for Stage-1. Semantic validation is performed only on an exact, 11-line AST micro-context.
- Zero-Trust Offline Semantic Validation: The implementation of a 4-bit quantized Meta Llama 3 (8B) model, which is fine-tuned using Low-Rank Adaptation (LoRA at 3,000 epochs) in an offline mode through the use of the Apple MLX framework on Unified Memory to protect against the loss of proprietary code.
- Deterministic Output Optimization: The development of deterministic few-shot prompting schema to enforce JSON schemata, thus eliminating complex reasoning loops to minimize processing time of the contextual information.

1.4 Paper Layout

The remainder of this manuscript is structured as follows. Section 2 reviews existing literature and identifies prevailing research gaps. Section 3 details the proposed hybrid system architecture, dataset descriptions, and the adopted methodologies. Section 4 presents the experimental setup, empirical results on the CredData benchmark, and a critical performance analysis regarding inference latency. Finally, Section 5 concludes the research and outlines future operational directives.

2 Literature Survey

Machine learning in static code analysis has been well researched as an approach for tackling the problem of false positive alerts. According to research conducted by Apostolidis et al. (2025) [1], the use of AI in static analysis was proven effective in filtering outputs generated by CppCheck software, utilizing a customized version of CodeBERT. Nonetheless, it was found that vulnerability prediction models developed by Apostolidis et al. [1]. lack line-level accuracy, thus making static tools vulnerable to the issue of too many false alarms. Additionally, in their systematic review of 130 mitigation efforts, Guo et al. (2023) [3] highlighted how the current generation of deep learning and NLP methods suffers from poor explainability, ambiguous definitions of concepts, and dataset bias. Lastly, in his work on the adaptation of LLMs, Jin (2025) [4] described various approaches, among which include base application, static augmentation, RAG, and hybrid fusion.

Recent efforts have tried to direct the attention of LLMs towards localized semantics to overcome context blindness. Li et al. (2025) [6] obtained a boost in their F1-score of 12.86% by directly injecting vulnerability steering vectors into LLM activation layers using actual data. However, current techniques for representation are mainly based on file semantics rather than identifying localized vulnerabilities. Kim et al. (2025) [5] further validated this issue by implementing reinforcement learning to identify contextual signals, highlighting how conventional LLM models inherently fail when faced with vague tasks requiring context extraction. Classical data flow models, as presented by Muske and Khedker (2015) [7], were able to accelerate processing speeds by reducing model check calls; however, they lacked detection accuracy because of path over-approximation.

Crucially, empirically derived tests have identified serious problems in the way LLM security benchmarks are being assessed. In a startling finding, Dozono et al. (2025) [2] found a devastating decline in accuracy from 56% in open source software to 34% in closed source commercial software, indicating that present day LLM security is overstated owing to contaminated training data sets. Lastly, moving LLM defenses solely to a reactive defense mecha-

nism makes systems highly vulnerable to prompt injection attacks, requiring pre-inference structural verification.

Together, these studies identify three key knowledge gaps, namely extreme token ineffectiveness during the analysis of the complete repository, “context blindness” arising out of the inability to isolate semantics at the line level, and zero-trust violations due to the transmission of proprietary code to cloud-based LLM APIs. Such limitations mandate a local hybrid framework.

3 Proposed System/Model

This architecture addresses the limitations associated with token depletion and data security of current LLM security products through its "cascading filter" mechanism that is performed purely locally.

3.1 Dataset Description

To empirically analyze the proposed algorithm, we used the CredData benchmark from the open-source community. The CredData dataset consists of around 27,000 samples of source code snippets created to test the capabilities of detection algorithms that deal with hardcoded credentials.

3.2 Schematic Layout of the proposed system/model

This process of pipelining takes a sequential approach to filtration. The raw source code goes through a directory filter first and then gets processed using the deterministic regex matcher. When the match is found, the context extractor extracts the immediate surroundings of the matching content. This content then gets sent for processing via the local Apple MLX engine with the fine-tuned Llama 3 model, producing only SAFE and CRITICAL JSON classifications.

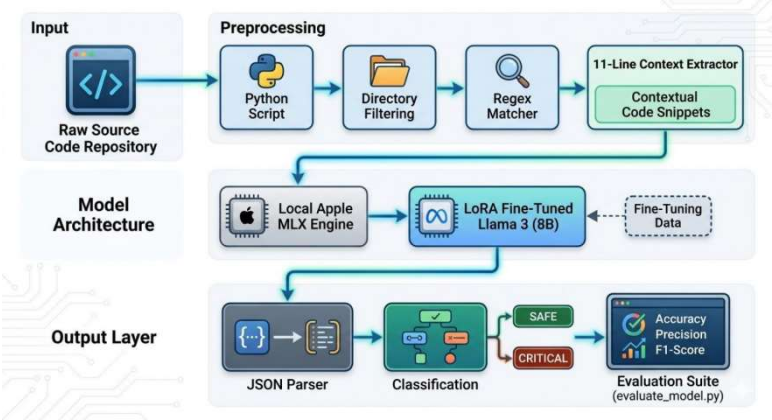

Fig. 1. Schematic Model Diagram of the Proposed Hybrid Architecture.

3.3 Methodologies Used

The proposed algorithm is based on the highly optimized two-step hybrid approach intended for deployment at a local level.

- **Step 1: Deterministic Heuristic Pre-Filtering.** The algorithm utilizes standard regular expressions to perform pre-filtering and identify secret candidates without waiting for the LLM to analyze the whole source code. Once a match is detected, the AST-based extractor extracts exactly 5 lines of code above and 5 lines below the matching code.

- **Stage 2: Semantic Validation Using Local LLM.** The isolated micro-context is analyzed using a local Meta Llama 3 (8B) Instruct model. For full offline processing and complete data privacy in the zero-trust manner, the inference engine utilizes the MLX Framework of Apple on the M-series chip using the concept of Unified Memory.

- **Parameter-Efficient Fine-Tuning (PEFT).** As part of fine-tuning to enable binary classification under extreme hardware constraints, the model underwent 4-bit quantization along with LoRA. LoRA was applied to certain attention layers (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj) using LoRA rank (r) = 8 and alpha = 8. After 3,000 epochs, the model reached stability when it learned how to explicitly analyze variable scope, dummy parameters, and mock data definitions of the AST snippet.

- **Optimization of Deterministic Outputs.** In order to address latency in inference, a rigid few-shot prompt template was devised. The inputting of highly deterministic examples before inference enabled the system to avoid any reasoning cycles and imposed a JSON schema for outputs that were categorized as either SAFE, REQUIRE_MANUAL_REVIEW, or CRITICAL.

4 Experimentation and Model Evaluation

For an accurate measure of the effectiveness of the hybrid architecture suggested, an empirical comparison test was undertaken, using industry-standard offline legacy scanners.

4.1 Experimental Setup

In order to ensure the strict enforcement of data privacy paradigm that is necessary for modern DevSecOps, all experiments were run locally and off-line using Apple Silicon (architecture M series) and Unified Memory.

For our testing, we used an open-source benchmark called CredData. For creating a comparison baseline, Gitleaks (regex based) and TruffleHog (offline mode) were run against the complete dataset of the repository. However, due to significant computational latency associated with local LLM inference, performance of the Hybrid Scanner's Stage-2 semantic validation was measured against a standardized 1,000-hit sample of the Regex filtered candidates. Metrics: Precision, Recall, F1-Score.

4.2 Results and Outputs

There is a clear difference in detection performance of the legacy static approach versus that of LLM-powered validation.

TruffleHog was not well-suited for use in this scenario since its offline mode showed a dismal Recall (0.0123) and F1-Score (0.0238). The high Precision (0.8600) value of Gitleaks was possible due to its very strict Regex pattern matching algorithm, but due to lack of semantics, the tool exhibited a dramatic decrease in Recall (0.2600), failing to detect about 74% of hardcoded secrets in the set.

Unlike baselines, the proposed Hybrid Llama 3/MLX model managed to address their issues with lack of context understanding. The only negative was a slightly decreased Precision rate compared to Gitleaks

at 0.8117; however, with a much higher Recall value of 0.9427 enabled by the ability to identify dummy values and test configuration files within the provided context, the model achieved 87.0% of accuracy and F1-Score of 0.8723.

Fig. 2. Precision, Recall, and F1-Score evaluation of the Hybrid Llama 3/MLX architecture against industry baselines.

4.3 Validation/System Performance Evaluation

Although the mathematically proven detection accuracy confirms the hypothesis about the absence of alert fatigue as a result of applying semantic context, the speed of implementation is an important engineering challenge.

According to inference logs, the elapsed time of execution was 12,702.3 seconds (about 3.5 hours) to process 1,000 hits of the sample size. Performing a sequential execution of LLM inference on raw Regex matches leads to a significant pipeline blocking bottleneck. However, since the Apple MLX framework locally ensures Zero-Trust privacy and runs all code execution on-device, performing iterative execution on an 8B parameter model is too slow.

4.4 Discussions on Contributions

Indeed, the outcomes have conclusively proven the architectural claims made in this study. With the deterministic Regex pre-filter in place, the pipeline managed to overcome the significant problems of token exhaustion and context window constraints that have been plaguing monolithic LLM repository scanners up until now. More specifically, using an AST micro-context of 11 lines was found to be highly effec-

tive when isolating the vulnerability semantics. As a result, even a simple 4-bit quantized Meta Llama 3 was able to reach 0.8723 F1-Score and overcome context blindness.

More importantly, the successful implementation of this pipeline through the use of the Apple MLX framework creates a mathematically sound model for Zero-Trust DevSecOps. This demonstrates that highly advanced semantic verification can indeed be done on the local machine itself, thus precluding the need for sending proprietary source code to an external cloud-based API. Nevertheless, the observed 3.5 hours required for processing 1,000 secret candidates makes the stark truth clear that although the semantic verification process is indeed quite reliable, it is presently impossible to use it for real-time CI/CD integration.

5 Conclusion and Future Scope

This work is successful in designing and validating a two-tiered hybrid secret scanner which was able to address DevSecOps fatigue. By using deterministic high-speed heuristic filters in conjunction with a locally-trained fine-tuned language model, this two-tiered secret scanning framework obtained 87.0% accuracy in the CredData benchmarking test suite. Through semantic validation, this system demonstrates the ability to discriminate real cryptographic keys from test dummy variables with high recall rate of 0.9427 while enforcing Zero Trust through offline-only processing.

In order for this framework to be transformed from a research-level architecture to a practical one that can be implemented in an enterprise setting, further engineering work must focus solely on reducing latency. The emphasis on research will move away from maximizing accuracy towards throughput, studying MLX batch processing APIs as well as employing model pruning techniques to decrease latency of inference per snippet. Furthermore, the ability of the model to generalize will be tested against the FPSecretBench corpus, ultimately resulting in development of a GitHub Action.

References

[1] Apostolidis, G. D., Kalouptsoglou, I., Siavvas, M., Kehagias, D., and Tzovaras, D. (2025) 'AI-Enhanced Static Analysis: Reducing False Alarms Using Large Language Models', 2025 IEEE International Conference on Smart Computing (SMARTCOMP).

[2] Dozono, K., Engesser, J., Hummelt, B., Roehm, T., and Pretschner, A. (2025) 'Should We Evaluate LLM Based Security Analysis Approaches on Open

Source Systems?', 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE).

[3] Guo, Z. et al. (2023) 'Mitigating False Positive Static Analysis Warnings: Progress, Challenges, and Opportunities', IEEE Transactions on Software Engineering, Vol. 49, No. 12, pp. 5154-5186.

[4] Jin, S. (2025) 'Large Language Models for Vulnerability Detection in Static Code Analysis: A Survey', 2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD).

[5] Kim, M., Caccia, L., Shi, Z., Pereira, M., Côté, M.-A., Yuan, X., and Sordoni, A. (2025) 'Learning to Extract Context for Context-Aware LLM Inference', Microsoft Research.

[6] Li, J. et al. (2025) 'Steering Large Language Models for Vulnerability Detection', 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).

[7] Muske, T. and Khedker, U. P. (2015) 'Efficient Elimination of False Positives using Static Analysis', 2015 IEEE.

[8] Schwarz, D. (2025) 'Countermind: A Multi-Layered Security Architecture for Large Language Models', arXiv preprint arXiv:2510.11837v1.